

CKA: Kustomize Basics (2025 Updates)

KodeKloud Complete Study Notes — Certified Kubernetes Administrator

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/lms.txt> Use this file to discover all available pages before exploring further.

Kustomize vs Helm

This guide compares Helm and Kustomize for modifying Kubernetes manifests, highlighting their features, advantages, and trade-offs to help choose the best solution for projects.

Before moving forward, it's important to examine an alternative tool to Kustomize: Helm. This guide provides a high-level overview of how Helm addresses the need to modify Kubernetes manifests on a per-environment basis. By understanding both tools and their trade-offs, you can choose the best solution for your project.

Helm Templating Fundamentals

Helm utilizes Go templating syntax to define variables within your Kubernetes manifests. For example, consider the template snippet below, which outlines a standard Deployment configuration. Notice the use of double curly braces to denote template variables:

```
```yaml theme={null} apiVersion: apps/v1 kind: Deployment metadata: name: {{ .Values.name }} spec: replicas: {{ .Values.replicaCount }} selector: matchLabels: app: {{ .Values.name }} template: metadata: labels: app: {{ .Values.name }} spec: containers: - name: {{ .Chart.Name }} image: "nginx:{{ .Values.image.tag }}"
```

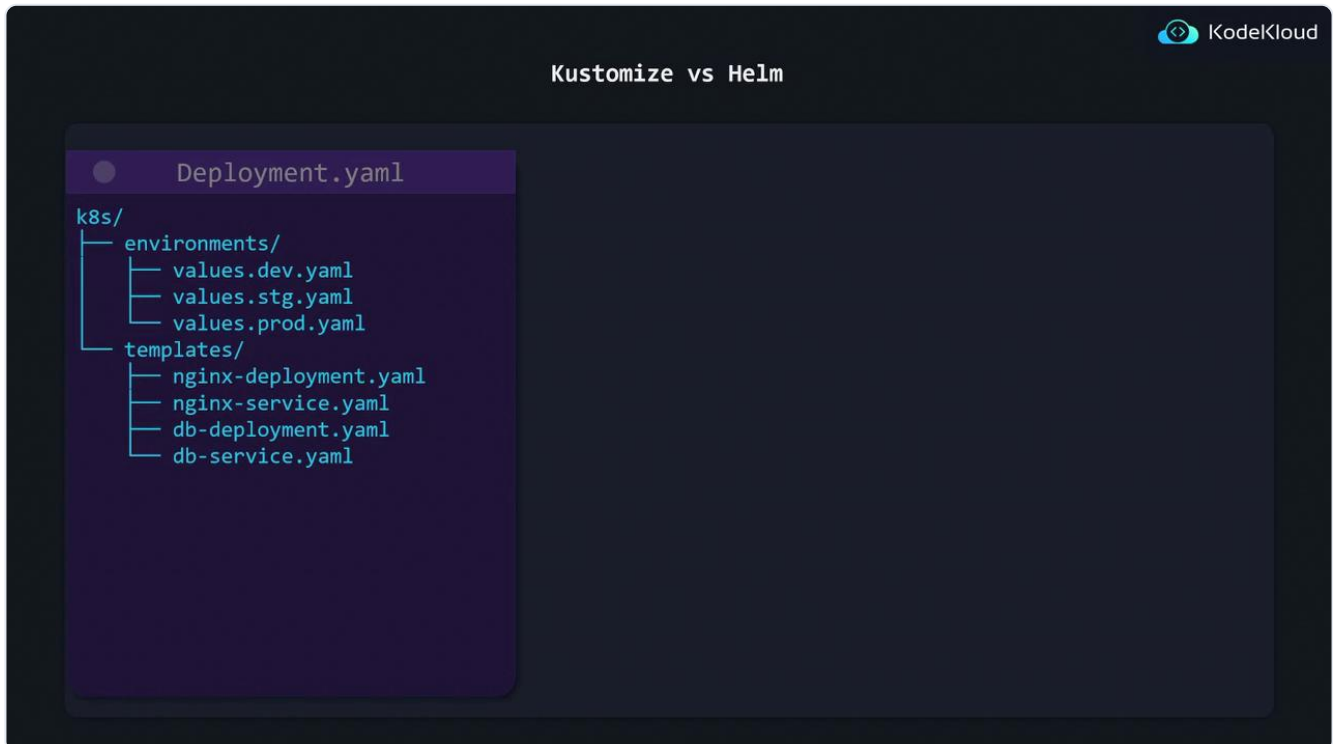
In this configuration, the replicas count (`replicaCount`) is not hardcoded. Instead, its value is external

```
```yaml theme={null}
replicaCount: 1
image:
  tag: "2.4.4"
```

When deploying your application, Helm merges the specified values into the template, replacing placeholders with concrete values.

Project Structure in a Traditional Helm Setup

A conventional Helm project organizes deployment templates in one directory and environment-specific configuration files in another. For instance, you might have a directory structure where the `environments` folder contains files such as `values.dev.yaml`, `values.stg.yaml`, and `values.prod.yaml`, while the `templates` folder stores the Kubernetes manifest files.



Below is an example of how the directory structure may be organized:

```
yaml theme={null} k8s/ environments/ values.dev.yaml values.stg.yaml values.prod.yaml templates/  
nginx-deployment.yaml nginx-service.yaml db-deployment.yaml db-service.yaml
```

When deploying your application, you select the relevant environment file to inject the proper values into your templates.

Advantages and Trade-offs of Helm

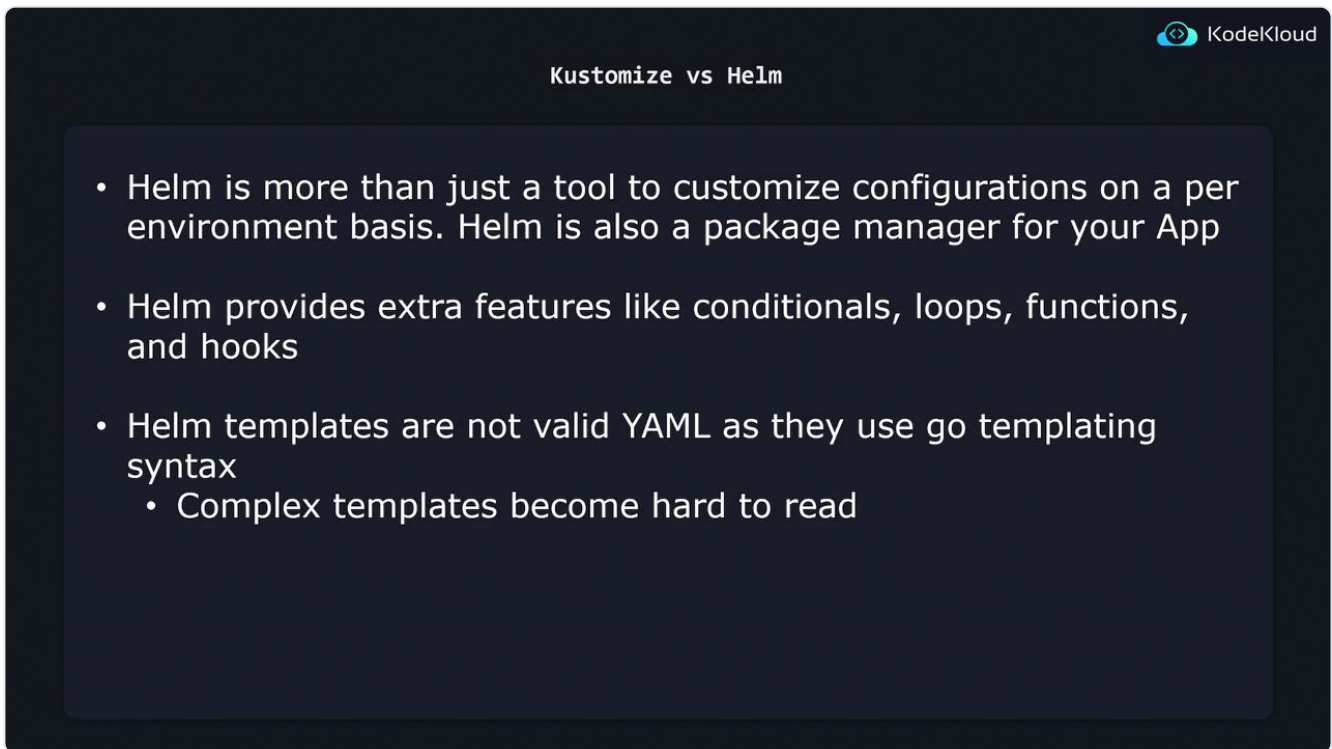
Helm is more than just a templating tool for environment-specific configurations—it serves as a complete package manager for Kubernetes applications. Similar to package managers like `yum` or `apt` in Linux, Helm provides additional features such as:

- Conditionals and loops
- Functions and hooks

While Helm's rich feature set offers significant flexibility, it also introduces complexity. The templates, which include Go templating syntax, are not strictly valid YAML and can be challenging to read and maintain initially.

A comparison between the two tools highlights that while Helm offers advanced capabilities like package management and flexible templating constructs, these benefits come with increased complexity. In contrast,

Kustomize relies on plain YAML for base configurations and overlays for environment modifications, which makes it simpler to read and maintain.

A dark-themed presentation slide titled "Kustomize vs Helm" with the KodeKloud logo in the top right corner. The slide contains a bulleted list of points comparing Helm and Kustomize.

Kustomize vs Helm

- Helm is more than just a tool to customize configurations on a per environment basis. Helm is also a package manager for your App
- Helm provides extra features like conditionals, loops, functions, and hooks
- Helm templates are not valid YAML as they use go templating syntax
 - Complex templates become hard to read

Choosing the Right Tool

When deciding between Helm and Kustomize, consider the specific requirements of your project and your tolerance for complexity:

- Choose **Helm** if you need advanced functionalities like package management, dynamic templating, and additional programming constructs.
- Choose **Kustomize** if you prefer a straightforward, plain YAML approach that improves readability and simplifies maintenance.

Be mindful that Helm templates can become difficult to manage as complexity increases. Always weigh the benefits against the added learning curve and potential maintenance overhead.

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

InstallationSetup

This guide explains how to install and set up Kustomize for customizing Kubernetes resource configurations.

In this guide, you'll learn how to install and set up Kustomize—a powerful tool for customizing Kubernetes resource configurations. Before proceeding, ensure that you have a running Kubernetes cluster and that [kubectl](#) is installed and configured on your local machine. Kustomize supports Linux, Windows, and macOS.

Installation Steps

The Kustomize team provides a convenient installation script that automatically detects your operating system and installs the appropriate version. To download and run this script, execute the following command in your terminal:

```
```bash theme={null} curl -s "https://raw.githubusercontent.com/kubernetes-sigs/kustomize/master/hack/install_kustomize.sh" | bash
```

After the script completes, verify the installation by checking the version of Kustomize with:

```
```bash theme={null}
kustomize version --short
```

You should see an output resembling the following:

```
text theme={null} {kustomize/v4.4.1 2021-11-11T23:36:27Z }
```

If you do not see the expected version output, it may indicate an installation issue or that your current terminal session has not updated the necessary environment variables.

Troubleshooting

If you encounter any issues during installation, try the following steps:

1. Close your current terminal session and reopen it.
2. Rerun the installation script to ensure that all components are set up correctly.

For more detailed troubleshooting and advanced configuration options, refer to the [Kustomize Documentation](#).

With Kustomize properly installed, you are now ready to explore its powerful features for managing Kubernetes configurations. Enjoy customizing your Kubernetes deployments!

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/lms.txt> Use this file to discover all available pages before exploring further.

Kustomize Output

This article explains how to deploy and delete Kubernetes resources using Kustomize in conjunction with the `kubectl` command.

In the previous lesson, we explored the Kustomize build command. Running this command processes all defined resources, applies the necessary transformations, and outputs the final configurations to your console. Remember, these configurations are not automatically deployed to your Kubernetes cluster. When you inspect your cluster using commands like `kubectl get pods`, `kubectl get deployments`, or `kubectl get services`, you'll notice that no resources have been created yet.

Kustomize focuses on building configurations. To deploy these configurations, you must combine it with the `kubectl apply` command.

Deploying Generated Configurations

To deploy your configurations, you can chain the Kustomize build command with the `kubectl apply` command using the Linux pipe (`|`). The command below demonstrates how to build and immediately apply the configurations from the `k8s/` directory:

```
```bash theme={null} $ kustomize build k8s/ | kubectl apply -f - service/nginx-loadbalancer-service created deployment.apps/nginx-deployment created
```

The Linux pipe utility takes the output from ``kustomize build k8s/`` and supplies it as the input to ``kubectl apply -f -``. Alternatively, the ``kubectl`` command includes native support for Kustomize. By using the ``-k`` flag, you can

```
```bash theme={null}
$ kubectl apply -k k8s/
service/nginx-loadbalancer-service created
deployment.apps/nginx-deployment created
```

Deleting Resources Using Kustomize

Removing resources works in a similar way to deploying them. You simply replace the word `apply` with `delete`. This section demonstrates how to delete resources with both methods.

Using the Pipe Method

The following command uses the pipe utility to delete the previously created resources:

```
```bash theme={null} $ kustomize build k8s/ | kubectl delete -f - service "nginx-loadbalancer-service" deleted
deployment.apps "nginx-deployment" deleted
```

### Using the Native kubectl Command

You can also use the `kubectl` command with the `-k` flag to delete the resources:

```
```bash theme={null}
$ kubectl delete -k k8s/
service "nginx-loadbalancer-service" deleted
deployment.apps "nginx-deployment" deleted
```

This article has demonstrated how to both apply and delete Kubernetes resources using Kustomize in combination with `kubectl`. For more details on using Kubernetes tools and best practices, refer to the [Kubernetes Documentation](#).

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/lms.txt> Use this file to discover all available pages before exploring further.

Managing Directories

This guide explains how to structure Kubernetes YAML files effectively using Kustomize for streamlined management and deployment workflows.

Organizing and managing your Kubernetes manifests across multiple directories can be streamlined with Kustomize. This guide explains how to structure your YAML files effectively, simplify deployment workflows, and maintain a clean configuration hierarchy for your clusters.

Basic Directory Structure Without Kustomize

Initially, you might store all your Kubernetes YAML files in a single directory (e.g., a directory named "k8s"). In this simple setup, you could have files such as:

- API deployment YAML file
- API service YAML file
- Database deployment YAML file
- Database service YAML file

To deploy these configurations, navigate to your "k8s" directory and run:

```
``bash theme={null} kubectl apply -f .
```

This method works well for a small number of files. However, as your application scales, you'll likely

```
## Organizing YAML Files into Subdirectories
```

A more structured approach is to organize your manifests into subdirectories. For instance, you can place

```
``bash theme={null}
kubectl apply -f k8s/api/
```

```
``bash theme={null} kubectl apply -f k8s/db/
```


While this method is functional, it may become cumbersome when dealing with numerous subdirectories, especially when the number of subdirectories grows.

Simplifying Deployment with Kustomize

Kustomize simplifies this process by letting you define a single `kustomization.yaml` file that aggregates all the resources.

```
```yaml
theme: {null}
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization

Kubernetes resources to be managed by Kustomize
resources:
 - api/api-depl.yaml
 - api/api-service.yaml
 - db/db-depl.yaml
 - db/db-service.yaml
```

With this configuration, deploy all resources from the root directory using one of the following commands:

```
```bash
theme={null} kustomize build k8s/ | kubectl apply -f -
```

Or leverage kubectl's native Kustomize support:

```
```bash
theme={null}
kubectl apply -k k8s/
```

These commands aggregate the YAML files specified in `kustomization.yaml` and deploy them simultaneously, eliminating the need to apply individual directories manually.

## Scaling with Multiple Directories

As your Kubernetes project grows, you might add more subdirectories for additional services like caches or messaging systems (e.g., Redis or Kafka). For instance, your root `kustomization.yaml` file might expand to include these new resources:

```
```yaml
theme: {null}
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
```

Kubernetes resources across multiple subdirectories

```
resources:
  - api/api-depl.yaml
  - api/api-service.yaml
  - db/db-depl.yaml
  - db/db-service.yaml
  - cache/redis-depl.yaml
  - cache/redis-service.yaml
  - cache/redis-config.yaml
  - kafka/kafka-depl.yaml
  - kafka/kafka-service.yaml
  - kafka/kafka-config.yaml
```

<Callout icon="lightbulb" color="#1CB2FE">

Consider breaking down the configurations into subdirectories with their own `kustomization.yaml` files

</Callout>

Using Individual kustomization.yaml Files in Subdirectories

For improved maintainability, create a separate `kustomization.yaml` file within each subdirectory (sub

```
``yaml theme={null}
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
```

```
resources:
  - db-depl.yaml
  - db-service.yaml
```

Then, update the root `kustomization.yaml` file to reference each subdirectory:

```
``yaml theme={null} apiVersion: kustomize.config.k8s.io/v1beta1 kind: Kustomization
```

```
resources: - api/ - db/ - cache/ - kafka/
```

Kustomize will recursively process each subdirectory's `kustomization.yaml`, aggregating all the resources

Deploying the Configurations

After setting up both the root and subdirectory-specific `kustomization.yaml` files, deployment becomes

```
``bash theme={null}
kustomize build k8s/ | kubectl apply -f -
```

Or, using kubectl's built-in Kustomize support:

```
bash theme={null} kubectl apply -k k8s/
```

Both methods compile all the manifests from your subdirectories and apply them in a single, efficient deployment process.

Conclusion

By organizing your Kubernetes YAML files into dedicated subdirectories and leveraging Kustomize with individual `kustomization.yaml` files, you can manage complex deployments more effectively. This approach not only cleans up your configuration hierarchy but also streamlines CI/CD pipelines by reducing repetitive commands.

For more insights on Kubernetes configuration management, consider exploring the following resources:

- [Kubernetes Basics](#)
- [Kubernetes Documentation](#)
- [Kustomize Overview](#)

Always test your Kustomize configurations in a staging environment before deploying to production. This practice helps ensure that your aggregated manifests work as expected across your Kubernetes clusters.

Documentation Index

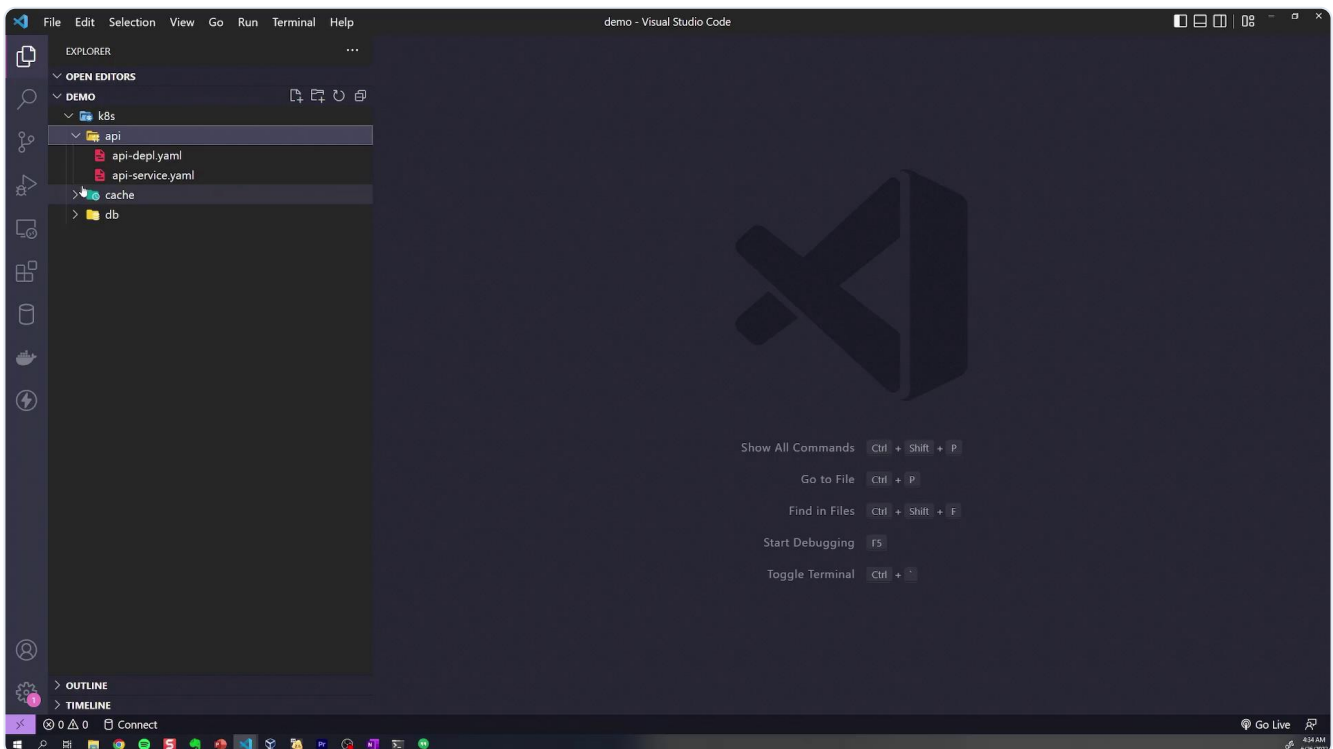
Fetch the complete documentation index at: <https://notes.kodekloud.com/lms.txt> Use this file to discover all available pages before exploring further.

Managing Directories Demo

This article explores managing Kubernetes manifest directories and introduces Kustomize for simplifying resource management and deployment.

In this lesson, we explore how to effectively manage directories containing Kubernetes manifests. The demonstration uses a structured "K8s" directory that holds all Kubernetes configurations organized into three subdirectories: one for the API, one for the cache (acting as a readers' database), and one for the MongoDB database.

When you open the K8s directory, you'll see three distinct folders. Each folder includes configuration files (YAML manifests) tailored for a specific component. For instance, the database folder contains the deployment YAML files for MongoDB, while the API and cache directories contain configurations for services such as ClusterIP or LoadBalancer services along with associated ConfigMaps.



Below is an excerpt showcasing a typical service configuration for the cache component:

```
```yaml theme={null} apiVersion: v1 kind: Service metadata: name: redis-cluster-ip-service spec: type: ClusterIP selector: component: redis ports: - port: 6379 targetPort: 6379
```

```
Before we introduce Kustomize, let's deploy these resources using the conventional approach without customizations.

```bash theme={null}
kubectl apply -f k8s/api
# Output:
# deployment.apps/api-deployment created
kubectl apply -f k8s/cache
# (Applies cache-related configurations)
```

You can also deploy all configurations in one command by appending multiple `-f` flags:

```
```bash theme={null} kubectl apply -f k8s/api kubectl apply -f k8s/cache kubectl apply -f k8s/db
```

To delete all resources simultaneously, you can run:

```
```bash theme={null}
kubectl delete -f k8s/db -f k8s/cache -f k8s/api
```

Using multiple `-f` flags simplifies bulk deployment and deletion. However, as your infrastructure grows, managing these commands can become cumbersome.

Simplifying Resource Management with Kustomize

Kustomize makes it easier to manage and customize your application configurations. Begin by creating a `kustomization.yaml` file in the root of your K8s directory. This file specifies the API version and kind required by Kustomize:

```
```yaml theme={null} apiVersion: kustomize.config.k8s.io/v1beta1 kind: Kustomization
```

Next, define the resources you wish to manage by listing the file paths relative to the `kustomization.yaml`

```
```yaml theme={null}
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
resources:
- api/api-depl.yaml
- api/api-service.yaml
- cache/redis-config.yaml
- cache/redis-depl.yaml
- cache/redis-service.yaml
- db/db-config.yaml
- db/db-depl.yaml
- db/db-service.yaml
```

You can then build the complete set of manifests using the Kustomize CLI:

```
```bash theme={null} kustomize build k8s/
```

```
This command outputs the final Kubernetes manifests, combining configurations from the API, cache, and data directories.

```bash theme={null}
kustomize build k8s/ | kubectl apply -f -
```

Alternatively, leverage the built-in support for Kustomize in kubectl with the `-k` flag:

```
```bash theme={null} kubectl apply -k k8s/
```

```
After applying the configurations, verify that the resources have been successfully created by checking the pods.

```bash theme={null}
kubectl get pods
```

Expected output:

```
```bash theme={null} NAME READY STATUS RESTARTS AGE api-deployment-64dd567b46-1mw4c 1/1 Running
0 27s db-deployment-657c8fbd8-vnjs7 1/1 Running 0 26s redis-deployment-587fd758cf-7pt57 1/1 Running 0 26s
```

```
<Callout icon="lightbulb" color="#1CB2FE">
 For quick troubleshooting, always verify your pods' status with `kubectl get pods` after deploying configurations.
</Callout>
```

## ## Advanced Directory Structuring with Kustomize

While maintaining a single `kustomization.yaml` in the root directory works for simple projects, a more structured approach is often preferred for complex setups.

### ### API Directory Configuration

In the API folder, create a `kustomization.yaml` that lists the API deployment and service manifests:

```
```yaml theme={null}
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
resources:
  - api-depl.yaml
  - api-service.yaml
```

Cache Directory Configuration

In the cache directory, set up the following `kustomization.yaml`:

```
```yaml theme={null} apiVersion: kustomize.config.k8s.io/v1beta1 kind: Kustomization resources:
- redis-config.yaml
- redis-depl.yaml
- redis-service.yaml
```

### ### Database Directory Configuration

Similarly, for the database directory, create a `kustomization.yaml`:

```
```yaml theme={null}
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
resources:
  - db-config.yaml
  - db-depl.yaml
  - db-service.yaml
```

Root Directory Aggregation

Finally, update the root `kustomization.yaml` to reference these subdirectories. When a directory is specified as a resource, Kustomize automatically searches for a `kustomization.yaml` file inside:

```
```yaml theme={null} apiVersion: kustomize.config.k8s.io/v1beta1 kind: Kustomization resources: - api/ - cache/ - db/
```

Before re-deploying, delete any previously applied resources:

```
```bash theme={null}
kubectl delete -f k8s/db -f k8s/cache -f k8s/api
```

Then, build the final configuration with Kustomize:

```
```bash theme={null} kustomize build k8s/
```

Review the output to ensure it meets your expectations, then apply the aggregated configuration:

```
```bash theme={null}
kubectl apply -k k8s/
```

The expected output should be similar to:

```
```bash theme={null} configmap/redis-credentials created service/api-service created service/db-service created
service/redis-cluster-ip-service created deployment.apps/api-deployment created deployment.apps/db-
deployment created deployment.apps/redis-deployment created
```

Verify the pods again:

```
```bash theme={null}
kubectl get pods
```

Expected output:

```
bash theme={null} NAME READY STATUS RESTARTS AGE api-deployment-64dd567b46-1mw4c 1/1 Running 0
27s db-deployment-657c8fbd8-vnjs7 1/1 Running 0 26s redis-deployment-587fd758cf-7pt57 1/1 Running
0 26s
```

This structured approach using Kustomize not only centralizes the management of your Kubernetes configurations but also offers a scalable solution for handling an expanding set of resources within your infrastructure.

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

Common Transformers

Learn to use Kustomize transformers for modifying Kubernetes configurations, focusing on common transformations for consistent resource management.

In this lesson, you will learn how to use Kustomize transformers to modify Kubernetes configurations. Kustomize supports several built-in transformers, and you can also create custom ones. Here, we focus on a subgroup known as Common Transformers.

Imagine you have multiple YAML files such as `deployment.yaml` and `service.yaml`. You might want to apply a common configuration—for example, adding a label like `"org: KodeKloud"` or appending `"-dev"` to resource names—across all these files. Manually updating each file in a production environment isn't scalable or efficient. Kustomize transformers offer a systematic way to make consistent changes across all resources.

Below are the original Kubernetes resource examples:

```
```yaml theme={null} apiVersion: apps/v1 kind: Deployment metadata: name: api-deployment spec: replicas: 1
selector: matchLabels: component: api template: metadata: labels: component: api spec: containers: - name:
nginx image: nginx
```

```
```yaml theme={null}
apiVersion: v1
kind: Service
metadata:
  name: db-service
spec:
  selector:
    component: db-depl
  ports:
    - protocol: "TCP"
      port: 27017
      targetPort: 27017
  type: LoadBalancer
```

After applying Kustomize transformations, the resources might look like this:

```
```yaml theme={null} apiVersion: apps/v1 kind: Deployment metadata: name: api-deployment-dev spec: replicas:
1 selector: matchLabels: component: api org: KodeKloud template: metadata: labels: component: api org:
KodeKloud spec: containers: - name: nginx image: nginx
```



```
```yaml theme={null}
apiVersion: v1
kind: Service
metadata:
  labels:
    org: KodeKloud
    name: api-service-dev
spec:
  selector:
    component: api
  ports:
  - protocol: "TCP"
    port: 80
    targetPort: 3000
  type: LoadBalancer
```

Kustomize transformers are essential for ensuring that your Kubernetes configurations remain consistent and manageable across various environments.

Common Transformation Methods

Below is an overview of common transformations available in Kustomize for managing Kubernetes resources:

1. Common Label Transformation

This transformer automatically adds the specified labels to all Kubernetes resources. You can define the labels in your `kustomization.yaml` file as shown below:

```
```yaml theme={null} commonLabels: org: KodeKloud
```

For example, a transformed Service resource would appear as:

```
```yaml theme={null}
apiVersion: v1
kind: Service
metadata:
  labels:
    org: KodeKloud
    name: api-service
spec:
  ports:
  - port: 80
    protocol: TCP
    targetPort: 3000
  selector:
    component: api
  type: LoadBalancer
```

2. Namespace Transformation

The namespace transformer assigns all Kubernetes resources to a specified namespace. By specifying the namespace in your `kustomization.yaml`, all resources will be modified accordingly. For example:

```
``yaml theme={null} namespace: lab
```

After this transformation, a Service resource might look like:

```
``yaml theme={null}
apiVersion: v1
kind: Service
metadata:
  annotations:
    branch: master
  labels:
    org: KodeKloud
    name: api-service
    namespace: lab
spec:
  ports:
    - port: 80
      protocol: TCP
      targetPort: 3000
  selector:
    component: api
    org: KodeKloud
  type: LoadBalancer
```

When applying namespace transformations, ensure that the specified namespace exists in your cluster to avoid deployment issues.

3. Name Prefix and Suffix Transformation

This transformer enables you to systematically add a prefix or suffix to resource names. For instance, to prepend "KodeKloud-" and append "-dev" to each resource name, include the following in your `kustomization.yaml`:

```
``yaml theme={null} namePrefix: KodeKloud- nameSuffix: -dev
```

After applying this configuration, a Service resource would be renamed to "KodeKloud-api-service-dev":

```
``yaml theme={null}
apiVersion: v1
kind: Service
metadata:
  name: KodeKloud-api-service-dev
spec:
  ports:
    - port: 80
      protocol: TCP
      targetPort: 3000
  selector:
    component: api
  type: LoadBalancer
```

4. Common Annotation Transformation

If you need to add specific annotations to all resources, use the common annotations transformer. By setting the annotations in your `kustomization.yaml`, each resource will automatically include them. For example:

```
``yaml theme={null} commonAnnotations: branch: master
```

This transformation results in a Service resource similar to:

```
``yaml theme={null}
apiVersion: v1
kind: Service
metadata:
  annotations:
    branch: master
  labels:
    org: KodeKloud
    name: api-service
    namespace: auth
spec:
  ports:
    - port: 80
      protocol: TCP
      targetPort: 3000
  selector:
    component: api
    org: KodeKloud
    type: LoadBalancer
```

Summary

The common transformations available in Kustomize include:

Transformation Type	Purpose
Common Label Transformation	Adds specified labels (e.g., org: KodeKloud) to all resources
Namespace Transformation	Assigns a specific namespace to all resources
Name Prefix and Suffix	Adds predetermined prefixes and suffixes to resource names
Common Annotation Transformation	Appends specific annotations (e.g., branch: master) to resources

These methods provide a scalable and systematic approach to maintaining consistent configurations across your Kubernetes resources.

Common Transformations

- `commonLabel` – adds a label to all Kubernetes resources
- `namePrefix/Suffix` – adds a common prefix-suffix to all resource names
- `Namespace` – adds a common namespace to all resources
- `commonAnnotations` – adds an annotation to all resources

In summary, Kustomize transformers offer a robust and error-resistant way to apply common configurations—such as labels, namespaces, name modifications, and annotations—to your Kubernetes resources, ensuring that your deployments remain consistent and manageable across various environments.

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/lms.txt> Use this file to discover all available pages before exploring further.

Image Transformers

Learn to use Kustomize's image transformer for updating Kubernetes deployment images without modifying manifests directly.

In this article, you will learn how to use Kustomize's image transformer to update the images used by your Kubernetes deployments without modifying your deployment manifests directly. This technique allows you to change either the entire image or just update its tag.

Example Deployment

Consider the following deployment that uses an Nginx container. This configuration is defined in a file named `deployment.yaml`:

```
```yaml theme={null} apiVersion: apps/v1 kind: Deployment metadata: name: web-deployment spec: replicas: 1 selector: matchLabels: component: web template: metadata: labels: component: web spec: containers: - name: web image: nginx
```

### ## Transforming the Image

To transform the image used by the deployment, create a `kustomization.yaml` file that specifies the desired transformation.

```
```yaml theme={null} images: - name: nginx newName: haproxy
```

Kustomize will scan all Kubernetes configuration files for containers that use the image named "nginx" and replace them with "haproxy".

The "name" property in the kustomization file is related to the image reference and is separate from the container name (which is "web" in the deployment manifest). This is a common point of confusion.

After applying this transformation, the updated deployment configuration will look like:

```
```yaml theme={null} apiVersion: apps/v1 kind: Deployment metadata: name: web-deployment spec: replicas: 1 selector: matchLabels: component: web template: metadata: labels: component: web spec: containers: - name: web image: haproxy
```

### ## Updating Only the Image Tag

If your goal is to update only the image tag, you can do so by using the `newTag`` property. For instance,

```
```yaml theme={null}
images:
  - name: nginx
    newTag: 2.4
```

After the transformation, the image reference in your deployment will update to include the new tag:

```
```yaml theme={null} apiVersion: apps/v1 kind: Deployment metadata: name: web-deployment spec: replicas: 1
selector: matchLabels: component: web template: metadata: labels: component: web spec: containers: - name:
web image: nginx:2.4
```

### ## Combining Image Name and Tag Changes

You can also combine both the `newName`` and `newTag`` properties if you wish to replace the image entirely

```
```yaml theme={null}
images:
  - name: nginx
    newName: haproxy
    newTag: 2.4
```

This configuration will ensure that your deployment now references the image `haproxy:2.4`.

By leveraging Kustomize's image transformer, you can manage image updates efficiently without directly altering your Kubernetes manifests. This process simplifies deployment updates and ensures consistency across your environments.

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/lms.txt> Use this file to discover all available pages before exploring further.

Transformers Demo

This guide demonstrates using transformation techniques with Kustomize, including managing Kubernetes configurations and applying common labels, namespaces, and image transformations.

In this guide, we demonstrate how to use common transformation techniques with Kustomize, including the image transformer. We also review the directory structure used in this demonstration to help you manage and organize Kubernetes configurations efficiently.

Directory Structure Overview

Assume you have a directory named "K8s" containing two subdirectories: "API" and "database".

- In the **API** directory, you will find:
 - API deployment YAML file (e.g., `api-depl.yaml`)
 - API service YAML file (e.g., `api-service.yaml`)
 - `kustomization.yaml`
- In the **database** directory, you will find:
 - Database deployment YAML file (e.g., `db-depl.yaml`)
 - Database service YAML file (e.g., `db-service.yaml`)
 - Config YAML file (e.g., `db-config.yaml`)
 - `kustomization.yaml`

Each subdirectory's `kustomization.yaml` is configured to import and manage all resources within that directory. For instance, the **API** directory's `kustomization.yaml` might look like this:

```
``yaml theme={null} apiVersion: kustomize.config.k8s.io/v1beta1 kind: Kustomization resources: - api-depl.yaml - api-service.yaml
```

And the **database** directory's `kustomization.yaml` imports the database configurations:

```
``yaml theme={null}
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
resources:
  - db-config.yaml
  - db-depl.yaml
  - db-service.yaml
```

At the root level, the `kustomization.yaml` aggregates the API and database configurations:

```
``yaml theme={null} apiVersion: kustomize.config.k8s.io/v1beta1 kind: Kustomization resources: - api/ - db/
```

Applying a Common Label

To add a common label to all resources, modify the root `kustomization.yaml` file. For example, to add the

1. Open the root `kustomization.yaml`.
2. Add the common label configuration.
3. Save the file and run the Kustomize build command.

When you run:

```
``bash theme={null}
kustomize build k8s/
```

the output will include all the resources with the label `department: engineering`. For example, a ConfigMap might appear as follows:

```
``yaml theme={null} apiVersion: v1 data: password: example username: root kind: ConfigMap metadata: labels:
department: engineering name: db-credentials
```

```
apiVersion: v1 kind: Service metadata: labels: department: engineering name: api-service spec: ports: - port: 80
protocol: TCP targetPort: 3000
```

All resources in both the API and database directories now include the `department: engineering` label.

Label Application in Subdirectories

When you add a common label within a subdirectory's `kustomization.yaml` rather than in the root file, the

```
``yaml theme={null}
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
resources:
  - api-depl.yaml
  - api-service.yaml
commonLabels:
  feature: api
```


the `feature: api` label is applied only to the resources listed in that file. This means that the API service will have both the root label (`department: engineering`) and the subdirectory-specific label (`feature: api`), while resources in the database directory will only have the global label.

A snippet of the resulting output could look like:

```
```yaml theme={null} apiVersion: v1 data: password: example username: root kind: ConfigMap metadata: labels:
department: engineering name: db-credentials
```

```
apiVersion: v1 kind: Service metadata: labels: department: engineering feature: api name: api-service spec:
ports: - port: 80 protocol: TCP targetPort: 3000
```

To differentiate between API and database resources further, you can add common labels specific to the database resources.

```
```yaml theme={null}
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
resources:
  - db-config.yaml
  - db-depl.yaml
  - db-service.yaml
commonLabels:
  feature: db
```

After building the configuration, database resources will include:

```
```yaml theme={null} metadata: labels: department: engineering feature: db
```

### ## Setting a Specific Namespace

To place all resources into a specific namespace, such as a debugging environment, add the `namespace` field to the configuration file.

```
```yaml theme={null}
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
resources:
  - api/
  - db/
commonLabels:
  department: engineering
namespace: debugging
```

After running the build, all resources from both API and database directories will be assigned the namespace `debugging`.

Adding Name Prefixes and Suffixes

If you want every object name to have a common prefix (e.g., "KodeKloud-") with folder-specific suffixes (like `-web` for API resources and `-storage` for database resources), follow these steps:

Adding a Global Name Prefix

Edit the root `kustomization.yaml` to include the prefix:

```
```yaml theme={null} apiVersion: kustomize.config.k8s.io/v1beta1 kind: Kustomization resources: - api/ - db/
commonLabels: department: engineering namespace: debugging namePrefix: KodeKloud-
```

After this change, the API deployment name might appear as `KodeKloud-api-deployment` (with an additional prefix).

### ### Adding Folder-Specific Name Suffixes

For the API subdirectory, add a name suffix in its `kustomization.yaml`:

```
```yaml theme={null}
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
resources:
  - api-depl.yaml
  - api-service.yaml
commonLabels:
  feature: api
nameSuffix: -web
```

For the database subdirectory, add a different name suffix:

```
```yaml theme={null} apiVersion: kustomize.config.k8s.io/v1beta1 kind: Kustomization resources: - db-config.yaml
- db-depl.yaml - db-service.yaml commonLabels: feature: db nameSuffix: -storage
```

After rebuilding, you will see that API resource names end with `-web` while database resource names end with `-storage`.

```
```yaml theme={null}
name: KodeKloud-api-deployment-web
```

and the database deployment might be:

```
```yaml theme={null} name: KodeKloud-db-deployment-storage
```

### ## Adding a Common Annotation

To apply a common annotation across every resource, add the following to your root `kustomization.yaml`:

```
```yaml theme={null}
commonAnnotations:
  logging: verbose
```

After rebuilding, every resource will include the annotation `logging: verbose`.

Image Transformation

Finally, let's look at how to update container images using the image transformer. Consider a scenario where a database deployment is set to use the Mongo image, but you want to replace it with Postgres using a specific tag.

In the database directory's `kustomization.yaml` (or in another appropriate configuration file), add an image transformer configuration like this:

```
``yaml theme={null} images: - name: mongo newName: postgres newTag: "4.2"
```

This configuration instructs Kustomize to search for any container image named "mongo" and update it to

<Callout icon="lightbulb" color="#1CB2FE">

Ensure you enclose the new tag in quotes to avoid type conversion issues (e.g., `newTag: "4.2"`). This

Error: accumulating resources: ... json: cannot unmarshal number into Go struct field Image.images.newT
</Callout>

After applying this transformer and running:

```
``bash theme={null}  
kustomize build k8s/
```

the database deployment's container image should update to:

```
yaml theme={null} image: postgres:4.2
```

while all other image configurations remain untouched.

Wrapping Up

This guide demonstrated how to:

- Organize Kubernetes configuration files into modular directories.
- Apply common labels, name prefixes, and suffixes globally or specifically in each subdirectory.
- Configure a namespace and common annotations.
- Use an image transformer to update container images seamlessly.

In the next section, try the lab exercises to gain hands-on experience with customizing Kustomize configurations and transformers.

Happy customizing!

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/lms.txt> Use this file to discover all available pages before exploring further.

Patches Intro

This lesson explains how patches enable targeted modifications of Kubernetes configurations, allowing precise updates to specific objects rather than applying global changes.

In this lesson, we explain how patches offer a surgical approach for modifying Kubernetes configurations. Unlike common transformers that are efficient for applying global configuration changes—such as adding labels or setting namespaces—patches allow you to target one or a few specific objects. For example, if you need to update the replica count in a particular deployment, a customized patch permits you to precisely match and change the targeted object.

Understanding Patch Components

To create a patch, you need to provide three essential parameters:

1. **Operation Type:** Determines the action performed on a resource. The most commonly used operations are:
 2. **add:** Inserts a new element. For instance, when adding a container to a list in a deployment.
 3. **remove:** Deletes an element, such as removing a container or a label.
 4. **replace:** Swaps an existing value with a new one. For example, changing the replica count from 5 to 10.
5. **Target:** Specifies the criteria to identify the exact Kubernetes object(s) to patch. You can filter objects based on properties like:
 6. kind
 7. version
 8. name
 9. namespace
 10. label selector
 11. annotation selector

These criteria can be combined to narrow down your selection.

1. **Value:** Represents the new value to be applied or used for replacement. Note that for removal operations, no value is provided since the intention is to delete the target property.

When updating configurations, ensure that your patch accurately targets the intended resource to prevent unintended changes.

Example 1: Changing the Deployment Name

Consider the following `deployment.yaml` file with an initial configuration:

```
```yaml theme={null} apiVersion: apps/v1 kind: Deployment metadata: name: api-deployment spec: replicas: 1 selector: matchLabels: component: api template: metadata: labels: component: api spec: containers: - name: nginx image: nginx
```

If you want to update `metadata.name` from `api-deployment` to `web-deployment`, you can use an `inline JS`

```
```yaml theme={null}
patches:
  - target:
      kind: Deployment
      name: api-deployment
    patch: |-
      - op: replace
        path: /metadata/name
        value: web-deployment
```

After applying this patch, the updated deployment configuration becomes:

```
```yaml theme={null} apiVersion: apps/v1 kind: Deployment metadata: name: web-deployment spec: replicas: 1 selector: matchLabels: component: api template: metadata: labels: component: api spec: containers: - name: nginx image: nginx
```

### ## Example 2: Updating the Replica Count

Let's review another scenario. Suppose the original deployment has one replica, and you want to `increase JS`

```
```yaml theme={null}
apiVersion: apps/v1
kind: Deployment
metadata:
  name: api-deployment
spec:
  replicas: 1
  selector:
    matchLabels:
      component: api
  template:
    metadata:
      labels:
        component: api
    spec:
      containers:
        - name: nginx
          image: nginx
```

Using an inline patch in your `kustomization.yaml`, update the replica count as follows:

```
``yaml theme={null} patches: - target: kind: Deployment name: api-deployment patch: |- - op: replace path: /spec/replicas value: 5
```

After patching, the final `deployment.yaml` is updated to:

```
``yaml theme={null}
apiVersion: apps/v1
kind: Deployment
metadata:
  name: api-deployment
spec:
  replicas: 5
  selector:
    matchLabels:
      component: api
  template:
    metadata:
      labels:
        component: api
    spec:
      containers:
        - name: nginx
          image: nginx
```

Patch Methods in Kustomize

Kustomize supports two primary methods for defining patches:

1. JSON 6902 Patch

This method specifies the target and lists the patch operations. An example using JSON 6902 patch is:

```
``yaml theme={null}
```

JSON 6902 Patch

```
kustomization: patches: - target: kind: Deployment name: api-deployment patch: - op: replace path: /spec/replicas value: 5
```

2. Strategic Merge Patch

This approach resembles standard Kubernetes configuration files. You provide only the parts of the configuration that you want to update.

```
```yaml theme={null}
Strategic Merge Patch
kustomization:
 patches:
 - patch: |-
 apiVersion: apps/v1
 kind: Deployment
 metadata:
 name: api-deployment
 spec:
 replicas: 5
```

Both methods effectively update Kubernetes resources. The choice between JSON 6902 and strategic merge patches is based on personal preference, with many opting for the readability of strategic merge patches.

## Conclusion

---

This lesson provided an overview of using patches with Kustomize to perform precise modifications on your Kubernetes configurations. By selecting the appropriate patch type, you can update labels, names, replica counts, and other fields with confidence and ease.

For additional details about patching and Kustomize, check out the [Kustomize documentation](#) and explore more in the [Kubernetes Documentation](#).

## Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/lms.txt> Use this file to discover all available pages before exploring further.

# Different Types of Patches

This guide explains how to define patches using JSON 6902 and strategic merge patch methods with inline and separate file approaches.

In this guide, we explain how to define patches using both JSON 6902 and strategic merge patch methods. There are two primary approaches to defining a patch:

1. Inline—where the patch is embedded directly within the kustomization.yaml file.
2. Separate File—where the patch is stored in an external YAML file, keeping the kustomization.yaml file clean and uncluttered.

Below are detailed examples for both methods.

## Inline Patch Definition

In the inline approach, the patch is directly embedded in your kustomization.yaml file. This method is ideal for simple or singular modifications. For example:

```
``yaml theme={null} Kustomization: patches: - target: kind: Deployment name: api-deployment patch: | - op: replace path: /spec/replicas value: 5
```



<Callout icon="lightbulb" color="#1CB2FE">

Use inline patches for quick adjustments when you have only a few changes to manage.

</Callout>

\*\*\*

### ## Separate File Patch Definition

Alternatively, you can store your patch in an external YAML file and reference it in your kustomization.yaml

In your kustomization.yaml file, reference the external patch file:

```
``yaml theme={null}
Kustomization:
 patches:
 - path: replica-patch.yaml
 target:
 kind: Deployment
 name: nginx-deployment
```

And here is how the external file named replica-patch.yaml might look:

```
``yaml theme={null} - op: replace path: /spec/replicas value: 5
```

<Callout icon="lightbulb" color="#1CB2FE">

Using a separate file for patches enhances readability and maintainability when working with complex configurations.

\*\*\*

### ## Strategic Merge Patches

You can apply the same methodologies when working with strategic merge patches. Below are examples for both

#### ### Inline Approach for Strategic Merge Patch

Embed the strategic merge patch directly into your configuration:

```
``yaml theme={null}
patches:
 - target:
 kind: Deployment
 name: api-deployment
 patch: |
 - op: replace
 path: /spec/replicas
 value: 5
```

### Separate File Approach for Strategic Merge Patch

Reference an external patch file in your kustomization.yaml:

```
``yaml theme={null} patches: - path: replica-patch.yaml target: kind: Deployment name: nginx-deployment
```

And in the external `replica-patch.yaml`:

```
```yaml theme={null}
- op: replace
  path: /spec/replicas
  value: 5
```

For a strategic merge patch defined as a complete resource patch in a single YAML block, consider this inline approach:

```
```yaml theme={null} patches: | - patch: | apiVersion: apps/v1 kind: Deployment metadata: name: api-deployment
spec: replicas: 5
```

Or reference the complete resource patch from an external file:

```
```yaml theme={null}
patches:
- replica-patch.yaml
```

Always ensure that your patch operations accurately reflect the intended changes to avoid potential misconfigurations.

Conclusion

Choose the inline method for simplicity when only a few patches are needed, or opt for the separate file approach to maintain a cleaner and more manageable `kustomization.yaml`—especially when dealing with multiple or complex patches.

For further details on patching strategies and best practices, refer to the [Kustomize documentation](#).

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

Patches Dictionary

Learn to update, add, and remove keys in Kubernetes Deployment configurations using JSON 6902 patches and strategic merge patches with practical examples.

In this article, you'll learn how to update, add, and remove keys in a Kubernetes Deployment configuration using both JSON 6902 patches and strategic merge patches. Each example starts with a Deployment that contains a label with the key "component" set to "api". The goal is to modify or update these labels as needed in each scenario.

Updating a Key in a Dictionary

Using a JSON 6902 Patch

Consider the following Deployment configuration:

```
```yaml theme={null} apiVersion: apps/v1 kind: Deployment metadata: name: api-deployment spec: replicas: 1 selector: matchLabels: component: api template: metadata: labels: component: api spec: containers: - name: nginx image: nginx
```

To update the label from `"component: api"` to `"component: web"` using a JSON 6902 patch, include this patch:

```
```yaml theme={null} kustomization: patches: - target: kind: Deployment name: api-deployment patch: |- - op: replace path: /spec/template/metadata/labels/component value: web
```

The patch navigates to the "component" key within the labels dictionary using the path `/spec/template/metadata/labels/component` and replaces its value with "web".

The JSON 6902 patch method provides precise control when updating complex configurations. Choose the patch type that best fits your needs.

Using a Strategic Merge Patch

Alternatively, you can update the label using a strategic merge patch stored in a separate file, for example, `label-patch.yaml`. Your main Deployment configuration remains unchanged, and your `kustomization.yaml` is updated as follows:

```
```yaml theme={null} kustomization: patches: - label-patch.yaml
```

Here is an example of the contents for `label-patch.yaml`:

```
```yaml theme={null}
apiVersion: apps/v1
kind: Deployment
metadata:
  name: api-deployment
spec:
  template:
    metadata:
      labels:
        component: web
```

Kustomize will merge this patch with the original Deployment configuration, resulting in an updated "component" label.

Adding a New Key to a Dictionary

Using a JSON 6902 Patch

Suppose you want to add a new label `"org"` with the value `"KodeKloud"` while keeping the original `"component: api"` label. Start with the following Deployment:

```
```yaml theme={null} apiVersion: apps/v1 kind: Deployment metadata: name: api-deployment spec: replicas: 1
selector: matchLabels: component: api template: metadata: labels: component: api spec: containers: - name:
nginx image: nginx
```

Then, add the following JSON 6902 patch in your `kustomization.yaml`:

```
```yaml theme={null}
kustomization:
  patches:
    - target:
        kind: Deployment
        name: api-deployment
      patch: |-
        - op: add
          path: /spec/template/metadata/labels/org
          value: KodeKloud
```

This patch uses the `add` operation to insert the new key `"org"` with the specified value into the labels dictionary.

Using a Strategic Merge Patch

For a strategic merge patch, create a separate file (e.g., `label-patch.yaml`) with the following content:

```
```yaml theme={null} apiVersion: apps/v1 kind: Deployment metadata: name: api-deployment spec: template:
metadata: labels: org: KodeKloud
```

Then, reference the patch file in your `kustomization.yaml`:

```
```yaml theme={null}
kustomization:
  patches:
    - label-patch.yaml
```

Kustomize automatically merges the patch with the existing configuration while preserving both labels: `"component: api"` and `"org: KodeKloud"`.

When adding new keys, always verify that the target dictionary exists to avoid runtime errors.

Removing a Key from a Dictionary

Using a JSON 6902 Patch

Assume you have a Deployment configuration that includes two labels:

```
```yaml theme={null} apiVersion: apps/v1 kind: Deployment metadata: name: api-deployment spec: replicas: 1
selector: matchLabels: component: api template: metadata: labels: org: KodeKloud component: api spec:
containers: - name: nginx image: nginx
```

To remove the `"org"` label using a JSON 6902 patch, modify your `kustomization.yaml` as follows

```
```yaml theme={null}
kustomization:
  patches:
    - target:
        kind: Deployment
        name: api-deployment
      patch: |-
        - op: remove
          path: /spec/template/metadata/labels/org
```

This patch navigates to the `"org"` key and removes it, leaving only the `"component"` label in place.

Using a Strategic Merge Patch

To remove the `"org"` label via a strategic merge patch, create a patch file (e.g., `label-patch.yaml`) with the following content:

```
```yaml theme={null} apiVersion: apps/v1 kind: Deployment metadata: name: api-deployment spec: template:
metadata: labels: org: null
```

Reference this patch file in your `kustomization.yaml`:

```
``yaml theme={null}
kustomization:
 patches:
 - label-patch.yaml
```

Kustomize interprets the `null` value as an instruction to remove the `"org"` label from the original configuration.

Ensure that you specify the correct path for removal operations to avoid inadvertently deleting other keys in the configuration.

---

With these examples, you now understand how to update, add, and remove keys in a Kubernetes Deployment configuration using both JSON 6902 patches and strategic merge patches. For more detailed information on Kubernetes configurations and patch strategies, visit the [Kubernetes Documentation](#).

## Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

# Patches list

This lesson demonstrates modifying list items in Kubernetes Deployment configurations using JSON 6902 patches and strategic merge patches.

In this lesson, we demonstrate how to modify list items in a Kubernetes Deployment configuration using both JSON 6902 patches and strategic merge patches. You can update, add, or remove containers in the deployment's container list. The examples below illustrate how to work with list indices and container names effectively.

Below is the base deployment configuration defining a single container named "nginx" using the "nginx" image. Notice that the containers section is a list, indicated by the dash (-) before the container definition.

```
```yaml theme={null} apiVersion: apps/v1 kind: Deployment metadata: name: api-deployment spec: replicas: 1 selector: matchLabels: component: api template: metadata: labels: component: api spec: containers: - name: nginx image: nginx
```

Replacing a Container in the List Using a JSON 6902 Patch

To update both the name and image of the container, use a JSON 6902 patch in your `kustomization.yaml` file:

****kustomization.yaml:****

```
```yaml theme={null}
patches:
 - target:
 kind: Deployment
 name: api-deployment
 patch: |-
 - op: replace
 path: /spec/template/spec/containers/0
 value:
 name: haproxy
 image: haproxy
```

The path `/spec/template/spec/containers/0` specifies that the first element of the containers list (index 0) will be replaced. Once applied, Kustomize updates the container, switching from "nginx" to "haproxy" for both the name and image.

Remember that list indices in YAML start at zero. Always check the index to target the correct container.

---

## Replacing a Container Using a Strategic Merge Patch

---

If you prefer modifying just a part of the container configuration (e.g., only the image), you can use a strategic merge patch. First, reference your patch file in the `kustomization.yaml`:

**kustomization.yaml:**

```
``yaml theme={null} patches: - label-patch.yaml
```

Then, in the `label-patch.yaml` file, specify the container by its name and provide the updated image:

```
label-patch.yaml:

``yaml theme={null}
apiVersion: apps/v1
kind: Deployment
metadata:
 name: api-deployment
spec:
 template:
 spec:
 containers:
 - name: nginx
 image: haproxy
```

This patch finds the container named "nginx" in the list and updates its image to "haproxy."

---

## Adding a Container to the List Using a JSON 6902 Patch

---

If you want to add a second container to your deployment, you can achieve this using a JSON 6902 patch. In the example below, a new container with the name and image "haproxy" is appended to the containers list. The dash (-) at the end of the path indicates that the container should be added at the end of the list.

**kustomization.yaml:**

```
``yaml theme={null} patches: - target: kind: Deployment name: api-deployment patch: |- - op: add path: /spec/
template/spec/containers/- value: name: haproxy image: haproxy
```



If the order of containers is important, you can specify an exact index; however, appending to the end is

After applying this patch, the deployment configuration will include both the original "nginx" container

```
```yaml theme={null}
apiVersion: apps/v1
kind: Deployment
metadata:
  name: api-deployment
spec:
  replicas: 1
  selector:
    matchLabels:
      component: api
  template:
    metadata:
      labels:
        component: api
    spec:
      containers:
        - name: nginx
          image: nginx
        - name: haproxy
          image: haproxy
```

Adding a Container Using a Strategic Merge Patch

Alternatively, you can add a container using a strategic merge patch. Suppose the original configuration contains a container named "web" using the "nginx" image and you wish to add another container. First, reference the patch file in `kustomization.yaml`:

kustomization.yaml:

```
```yaml theme={null} patches: - label-patch.yaml
```

Then, list the additional container in the `label-patch.yaml` file:

```
label-patch.yaml:

```yaml theme={null}
apiVersion: apps/v1
kind: Deployment
metadata:
  name: api-deployment
spec:
  template:
    spec:
      containers:
        - name: haproxy
          image: haproxy
```

When merged, the resulting configuration will include both the original "web" container and the new "haproxy" container.

Deleting a Container from the List Using a JSON 6902 Patch

Imagine a scenario where your Deployment originally includes two containers: one named "web" with the "nginx" image and another named "database" with the "mongo" image.

Original Deployment:

```
```yaml theme={null} apiVersion: apps/v1 kind: Deployment metadata: name: api-deployment spec: replicas: 1 selector: matchLabels: component: api template: metadata: labels: component: api spec: containers: - name: web image: nginx - name: database image: mongo
```

To remove the "database" container (which is at index 1), use the following JSON 6902 patch:

```
kustomization.yaml:

```yaml theme={null}
patches:
  - target:
      kind: Deployment
      name: api-deployment
    patch: |-
      - op: remove
        path: /spec/template/spec/containers/1
```

After applying this patch, the deployment will only contain the "web" container.

Ensure that you use the correct index when attempting to remove a container. An incorrect index could lead to unexpected modifications in your deployment configuration.

Deleting a Container Using a Strategic Merge Patch

You can also delete a container using a strategic merge patch with the delete directive. Given the initial configuration that includes both "web" and "database" containers, create a patch that instructs Kustomize to remove the "database" container:

kustomization.yaml:

```
```yaml theme={null} patches: - label-patch.yaml
```

```
label-patch.yaml:

```yaml theme={null}
apiVersion: apps/v1
kind: Deployment
metadata:
  name: api-deployment
spec:
  template:
    spec:
      containers:
        - $patch: delete
          name: database
```

In this patch file, the "\$patch: delete" directive targets the container named "database" and removes it from the list. Consequently, the final configuration will only include the "web" container.

This lesson detailed multiple methods for manipulating list items in Kubernetes Deployments using both JSON 6902 patches and strategic merge patches. For more information on Kubernetes patch strategies, refer to the [Kubernetes Documentation](#).

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

Overlays

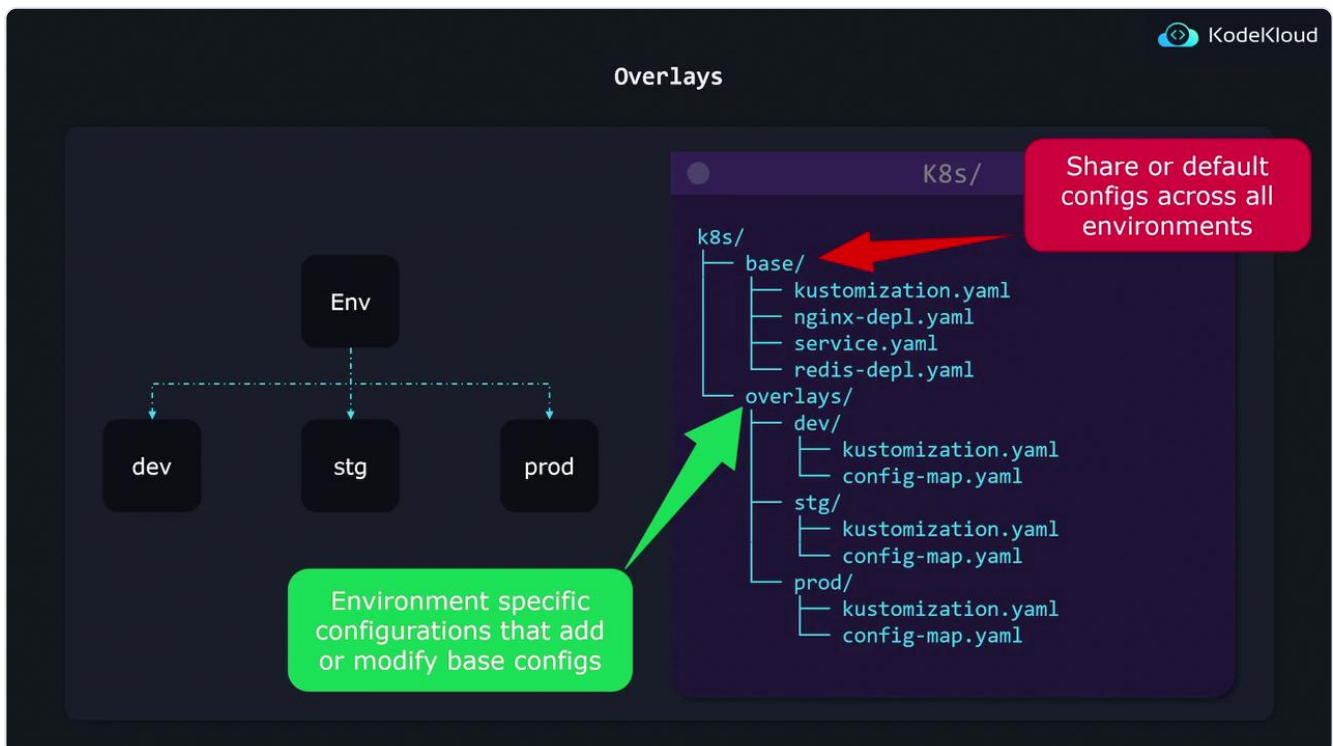
Overlays in Kustomize allow customization of base Kubernetes configurations for different environments like development, staging, and production.

Overlays in Kustomize allow you to customize a base Kubernetes configuration on a per-environment basis. This method is particularly useful for environments such as development, staging, and production, where you need to apply environment-specific adjustments to shared configurations.

Kustomize projects are typically organized into two main sections:

1. **Base Configuration:** Contains all shared and default Kubernetes resource definitions.
2. **Overlay Directories:** Each environment (e.g., dev, stg, prod) has its own overlay folder with patches to modify the base configuration as needed.

Below is a diagram illustrating a common directory structure for managing these configurations:



In this setup, the base folder holds the shared resource files, while each overlay folder contains a `kustomization.yaml` that references the shared resources in the base along with overlays (patches or additional resources) specific to that environment.

Base Configuration Example

Imagine that you have an `nginx-deployment.yaml` file within your base folder with a replica count set to 1. The corresponding `kustomization.yaml` in the base folder might look like this:

```
``yaml theme={null}
```

base/kustomization.yaml

resources: - nginx-depl.yaml - service.yaml - redis-depl.yaml

And the `nginx-depl.yaml` file is defined as follows:

```
``yaml theme={null}
# base/nginx-depl.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
spec:
  replicas: 1
```

Creating Overlays

Development Overlay

To create an overlay for the development environment, you would set up a `kustomization.yaml` file in the `dev` overlay folder. This file references the base configuration and includes a patch to update the replica count:

```
``yaml theme={null}
```

overlays/dev/kustomization.yaml

bases: - ../../base patch: |- - op: replace path: /spec/replicas value: 2

```

In this overlay, the `bases` property points to the shared base resources using the relative path ../../base.

### Production Overlay

Similarly, to tailor the configuration for production, the overlay can reference the same base while applying a patch to change the replica count.

```yaml theme={null}
overlays/prod/kustomization.yaml
bases:
- ../../base
patch: |-
- op: replace
 path: /spec/replicas
 value: 3

```

This overlay increases the replica count to 3 for production.

### Adding New Resources in Overlays

Overlays can also introduce new resources that don't exist in the base configuration. For instance, if you want to add a production-specific Grafana deployment, you can include its YAML file in the production overlay:

```
```yaml theme={null}
```

overlays/prod/kustomization.yaml

```
bases: - ../../base resources: - grafana-depl.yaml patch: |- - op: replace path: /spec/replicas value: 2 ```
```

In this configuration, the overlay imports the base resources, applies a patch to change the replica count for the existing deployment, and adds a new Grafana deployment.

Kustomize is flexible in the way you structure your configurations. While the base can be organized into subdirectories based on features, the overlay directories do not need to mirror that structure. The critical factor is correctly referencing the shared resources in the appropriate `kustomization.yaml` file.

Below is another diagram that provides a more detailed look at the directory structure using Kustomize, showing both the base and overlay directories across different environments:



```
k8s/
├── base/
│   ├── kustomization.yaml
│   ├── db/
│   │   ├── db-depl.yaml
│   │   ├── db-service.yaml
│   │   └── kustomization.yaml
│   └── api/
│       ├── api-depl.yaml
│       ├── api-service.yaml
│       └── kustomization.yaml
└── overlays/
    ├── dev/
    │   ├── kustomization.yaml
    │   ├── db/
    │   │   ├── db-patch.yaml
    │   │   └── kustomization.yaml
    │   └── api/
    │       ├── api-patch.yaml
    │       └── kustomization.yaml
    └── prod/
        ├── kustomization.yaml
        ├── db/
        │   ├── db-patch.yaml
        │   └── kustomization.yaml
        └── api/
            ├── api-patch.yaml
            └── kustomization.yaml
```

Summary

Overlays in Kustomize enable you to:

- Import and reuse a base configuration containing shared resources.
- Apply environment-specific patches to adjust base resources such as replica counts.
- Introduce new resources within an overlay without affecting the base configuration.

This approach helps maintain a clean separation between shared configurations and environment-specific customizations while taking full advantage of Kustomize's powerful features for managing Kubernetes deployments.

For further information on Kubernetes configuration management, consider exploring additional resources in the [Kubernetes Documentation](#) and [Kustomize GitHub repository](#).

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/lms.txt> Use this file to discover all available pages before exploring further.

Components

This article explains Kustomize components for defining reusable configuration logic to minimize duplication and manage optional features across multiple overlays.

In this lesson, you will learn about a powerful Kustomize feature called components. Components allow you to define reusable configuration logic that can be seamlessly integrated into multiple overlays. This approach is especially beneficial for applications that support optional features activated only in selected overlays.

For instance, if you have common configurations that apply to every overlay, you would include them in your Base configuration. However, if specific features need to be enabled only in particular overlays, duplicating the configurations in each overlay quickly becomes unscalable and error-prone. Components address this challenge by letting you define the configuration once and reuse it where needed, minimizing duplication and preventing configuration drift.

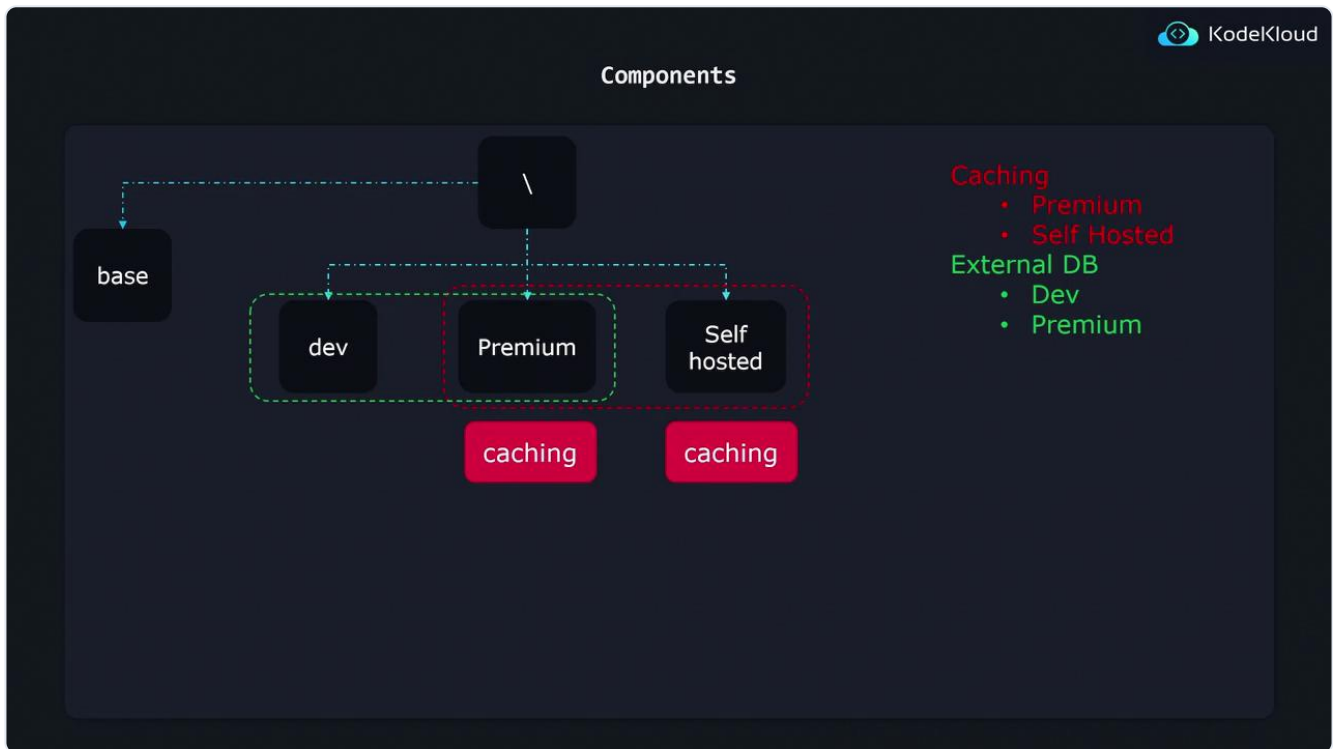
Visual Example

Imagine deploying an application in three variations: Development, Premium (for premium customers), and Self-Hosted (for customers who manage their own hosting). These variations correspond to three overlay folders, which all reference a shared Base configuration folder.

Suppose the application offers two optional features: caching and an external Postgres database. The caching feature, which includes configuration details for a Redis database, should only be applied to the Premium and Self-Hosted overlays. In contrast, the external database is only relevant for the Development and Premium overlays.

Placing the caching configuration in the Base folder would inadvertently activate it across all overlays, while copying it individually into the Premium and Self-Hosted overlays risks inconsistencies during future updates. Components solve this problem by encapsulating the caching configuration in a reusable block that is imported only by the overlays that require it.

The overall project hierarchy is structured so that the Base configuration is common to all overlays, while each overlay selectively includes components for features like caching or an external database.



Organizing Your Project Structure

To effectively implement components, consider adding a dedicated folder for components in your project structure. A typical folder layout may resemble the following structure:

```

``text theme={null} k8s/ |—— base/ | |—— kustomization.yaml | |—— api-depl.yaml |——
components/ | |—— caching/ | | |—— kustomization.yaml | | |—— deployment-patch.yaml | |
|—— redis-depl.yaml | |—— db/ | |—— kustomization.yaml | |—— deployment-patch.yaml | |——
postgres-depl.yaml | |—— overlays/ |—— dev/ | |—— kustomization.yaml |—— premium/ | |——
kustomization.yaml | |—— standalone/ | |—— kustomization.yaml
  
```

In the structure above, the `components` folder houses dedicated subdirectories for each optional feature.

<Callout icon="lightbulb" color="#1CB2FE">

A well-organized project differentiates between the immutable Base configuration and the flexible environment-specific configuration.

</Callout>

<Frame>

!The image is a flowchart titled "Components," showing a hierarchical structure with nodes labeled "Base," "Optional," and "Environment-Specific."

</Frame>

Implementing a Component

Let's focus on the database component. Within the `db` folder, the following three files are present:

```
* `kustomization.yaml`
* `deployment-patch.yaml`
* `postgres-depl.yaml`
```

The `postgres-depl.yaml` file defines the deployment for a Postgres database, which is the sole required resource.

```
```yaml theme={null}
apiVersion: apps/v1
kind: Deployment
metadata:
 name: postgres-deployment
spec:
 replicas: 1
 selector:
 matchLabels:
 component: postgres
 template:
 metadata:
 labels:
 component: postgres
 spec:
 containers:
 - name: postgres
 image: postgres
```

## The Component's kustomization.yaml

The `kustomization.yaml` file inside the database component is slightly different from standard configuration files. It indicates that this is a Component:

```
```yaml theme={null} apiVersion: kustomize.config.k8s.io/v1alpha1 kind: Component resources: - postgres-depl.yaml secretGenerator: - name: postgres-cred literals: - password=postgres123 patches: - deployment-patch.yaml
```

This file accomplishes the following:

- * Imports the Postgres deployment defined in `postgres-depl.yaml`.
- * Generates a secret named `postgres-cred` containing the database password.
- * Applies patches from `deployment-patch.yaml` to update the base configuration (for example, by integrating the database component into the API deployment).

The Deployment Patch

The `deployment-patch.yaml` file constitutes a strategic merge patch that updates the base API deployment.

```
```yaml
theme: {null}
apiVersion: apps/v1
kind: Deployment
metadata:
 name: api-deployment
spec:
 template:
 spec:
 containers:
 - name: api
 env:
 - name: DB_PASSWORD
 valueFrom:
 secretKeyRef:
 name: postgres-cred
 key: password
```
```

Importing Components in Overlays

To activate a component in an overlay, include its reference in the overlay's `kustomization.yaml` file. For example, the `kustomization.yaml` in the `dev` overlay might be configured as follows:

```
yaml theme={null} bases: - ../../base components: - ../../components/db
```

In this configuration:

- The base configuration is imported first.
- The database component is then imported, enabling the external database feature for that overlay.

This same import pattern can be applied to any overlay needing external database functionality or caching, ensuring that optional features are defined once and consistently reused across all environments.

By leveraging components in Kustomize, you can maintain consistency and reduce redundancy by defining optional features only once. This modularity not only promotes reusability but also simplifies managing configurations across multiple overlays.